Matrixdock XAUM Audit

audited by Asymptotic

August 11, 2025

Summary

Asymptotic conducted a security audit of Matrixdock XAUM contract. The audit identified 1 medium-severity, 1 low-severity, and 4 advisory issues. The medium-severity and low-severity issues have been remediated. The advisory issues have been remediated or acknowledged.

- Initial commit: `https://github.com/Matrixdock-RWA/RWA-Contracts/commit/dd655f3`
- Final commit: `https://github.com/Matrixdock-RWA/RWA-Contracts/commit/0cbee7b`
- Audited directories:
  `xaum-sui/packages/minter`
  `xaum-sui/packages/mtoken`
  `xaum-sui/packages/xaum`

Legend

Issue severity

- **Critical** — Vulnerabilities which allow account takeovers or stealing significant funds, along with being easy to exploit.
- **High** — Vulnerabilities which either can have significant impact but are hard to exploit, or are easy to exploit but have more limited impact.
- **Medium** — Moderate risks with notable but limited impact.
- **Low** — Minor issues with minimal security implications.
- **Advisory** — Informational findings for security/code improvements.

Legal Disclaimer

Terms and Conditions and Liability

This report is subject to the terms and conditions—including liability limitations—established between Asymptotic and the paying entity. By sharing code with Asymptotic, developers acknowledge and agree to these conditions.

Scope

This security audit report focuses specifically on reviewing the Move smart contract code. The audit does not analyze or make any claims about other components of the system, including but not limited to:

- Frontend applications and user interfaces
- Backend services and infrastructure
- Off-chain components and integrations
- Deployment procedures and operational security
- Third-party dependencies and external services

Limitations

The findings and recommendations in this report are limited to the Move code implementation and its immediate interactions within the Sui blockchain environment.

M-1: Failure in transfer_ownership

Severity: Medium

Description

Both request_transfer_ownership in mtoken and transfer_ownership in minter check that assert!(upgrade_cap.package().to_address() == state.package_address(), EUpgradeCapInvalid);. upgrade_cap.package() returns the latest version of the package, while state.package_address() return the address of the original package. These will not be equal after the first package upgrade.

Recommendation

I think the easies fix is to store the id of the UpgradeCap in the State object, and then check that in request_transfer_ownership/transfer_ownership.

Remediation

✓ **Commit:** 7988322

L-1: Possible ambiguity in accepted_by_a/accepted_by_b

Severity: Low

Description

accepted_by_a and accepted_by_b are Table¡address, bool¿, with keys the addresses of coin types. It is possible for multiple coin types to be in the same package, and thus share the same address.

Recommendation

Use Table¡TypeName, bool¿ instead.

Remediation

✓ **Commit:** f59bd18

A-1: No public entry

Severity: Advisory

Description

Functions should be either public or entry, not both. request_to_mint and request_to_redeem in minter are both.

Recommendation

[No specific recommendation provided]

Remediation

✓ **Commit:** e2321a7

A-2: Ambiguous Event Emission in Delayed Operations

Severity: Advisory

Description

In mtoken, both request_transfer_ownership/execute_transfer_ownership, request_set_operator/execute_set_o
, request_set_revoker/execute_set_revoker, request_set_delay/execute_set_delay, and request_mint_to/execute
emit the same event when called. This can creates ambiguity for off-chain systems trying to track actual ownership changes.

Recommendation

Emit distinct events for each stage of the ownership transfer process: - One for the request (e.g., OwnershipTransferRequested) - Another for the execution (e.g., OwnershipTransferred) This will ensure clear, unambiguous tracking and reduce risks related to off-chain misinterpretation.

Remediation

Acknowledged: the team confirmed that their backend will handle this appropriately, so they will keep the current implementation.

A-3: Missing Zero-Amount Validation in request_to_mint and request_to_redeem

Severity: Advisory

Description

In minter, both request_to_mint and request_to_redeem functions accept requests without checking that the amount amount ¿ 0. This omission allows zero-value operations to pass through.

Recommendation

Add an assertion to validate that the input amount is strictly greater than zero (amount ¿ 0). This ensures functional correctness and prevents pointless or malicious transactions.

Remediation

Acknowledged: the team confirmed that their backend will handle this appropriately, so they will keep the current implementation.

A-4: Missing Validation for Destination Token in mint Operation

Severity: Advisory

Description

In request_to_mint and request_to_redeem flow, the source token is properly validated via accepted_by_a/accepted_by_b, ensuring only whitelisted tokens can be used as input. However, the destination token is not validated.

Recommendation

Introduce a validation step for the destination token using accepted_by_b for request_to_mint and accepted_by_a for request_to_redeem.

Remediation

Acknowledged: the team confirmed that their backend will handle this appropriately, so they will keep the current implementation.

About the Auditor

Asymptotic provides a white-glove, formal-verification-based auditing service for Sui smart contracts.

Lead Auditor
Andrei Stefanescu – Chief Scientist

- Led verification of AWS cryptographic algorithms using SAW at Galois
- Created first Ethereum smart contract verification tool
- Three-time International Math Olympiad silver medalist
- PhD in Computer Science from UIUC with David J. Kuck Outstanding Thesis Award
- ACM SIGPLAN Distinguished Paper Award at OOPSLA 2016
- Pioneered K Framework for programming language semantics and verification